CHARUSAT UNIVERSITY

DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH (DEPSTAR)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
ACADEMIC YEAR 2026-27 [ODD SEMESTER]

LAB MANUAL

WEB DEVELOPMENT FRAMEWORKS (ITUE203)

Student Name:	**Krupanshu Patel**
Student ID / Roll No:	**25DCS088**
Semester & Branch:	3rd Semester, B.Tech CSE
Course Code:	ITUE203 (Web Development Frameworks)
GitHub Repository:	StudentHub-Portal
Repository URL:	https://github.com/krupanshupatel2347/StudentHub-Portal

Practical 1: Project Initiation, Requirement Analysis, Sitemap, Wireframe, and GitHub Setup

Problem Definition:	Initiate the semester-long StudentHub portal by identifying the problem scope, user roles, key modules, navigation flow, and minimum 10 pages. Create a sitemap, low-fidelity wireframe, project folder structure, README file, and GitHub repository.
CO Mapping:	CO1
Key Skills:	Requirement analysis, sitemap design, wireframing, Git/GitHub, README writing, folder structuring
Tools & Tech:	VS Code, Draw.io/Figma, Git, GitHub

GitHub Code Link: https://github.com/krupanshupatel2347/StudentHub-Portal

Sitemap & Navigation Flow (docs/sitemap.txt):

```
StudentHub Portal – Sitemap & Navigation Flow
=============================================
Student Name: Krupanshu Patel (25DCS088)

1. Home (Home.html)
   ├── About Us (About.html)
   ├── Academic Programs (Courses.html)
   ├── Admissions (Admission.html)
   ├── Study Materials (StudyMaterial.html)
   ├── Class Timetable (Timetable.html)
   ├── Campus Events (event.html)
   ├── Frequently Asked Questions (FAQ.html)
   ├── Support & Helpdesk (Help.html)
   ├── Student Registration (Signup.html)
   └── Account Login (Login.html)
        ├── Student Dashboard / Home
        └── Admin Dashboard (admin_dashboard.php)
             ├── Student Management CRUD (students.php)
             ├── Event Management CRUD (events.php)
             ├── AJAX Event API Management (index.html)
             └── System Audit Logs
```

Conclusion:

In this practical, the foundational requirements, user personas, sitemap, navigation hierarchy, and directory structure for the StudentHub portal were planned and initiated. A Git version-controlled repository was setup on GitHub with clear documentation.

Practical 2: Semantic HTML5 Pages with Accessibility-Ready Structure

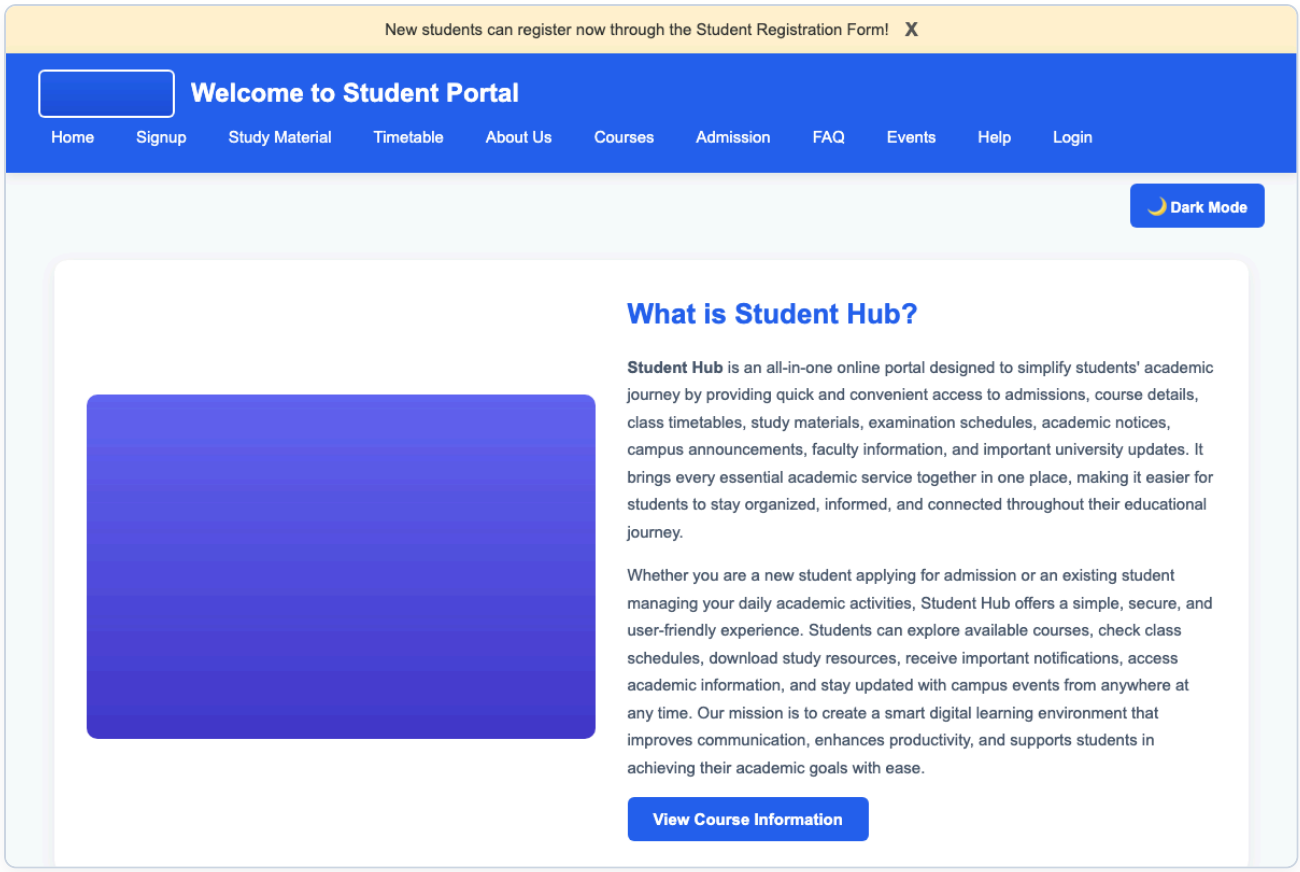
Problem Definition:	Develop static HTML5 skeletons for at least 10 StudentHub pages such as Home, About, Register, Login, Dashboard, Events, Profile, Contact, Admin, FAQ, and Feedback. Use semantic tags and accessibility-friendly structure.
CO Mapping:	CO1, CO2
Key Skills:	HTML5 semantic elements, accessibility basics (ARIA, alt text, form labels), page linking, navigation hierarchy

GitHub Code Link: https://github.com/krupanshupatel2347/StudentHub-Portal/blob/main/Home.html

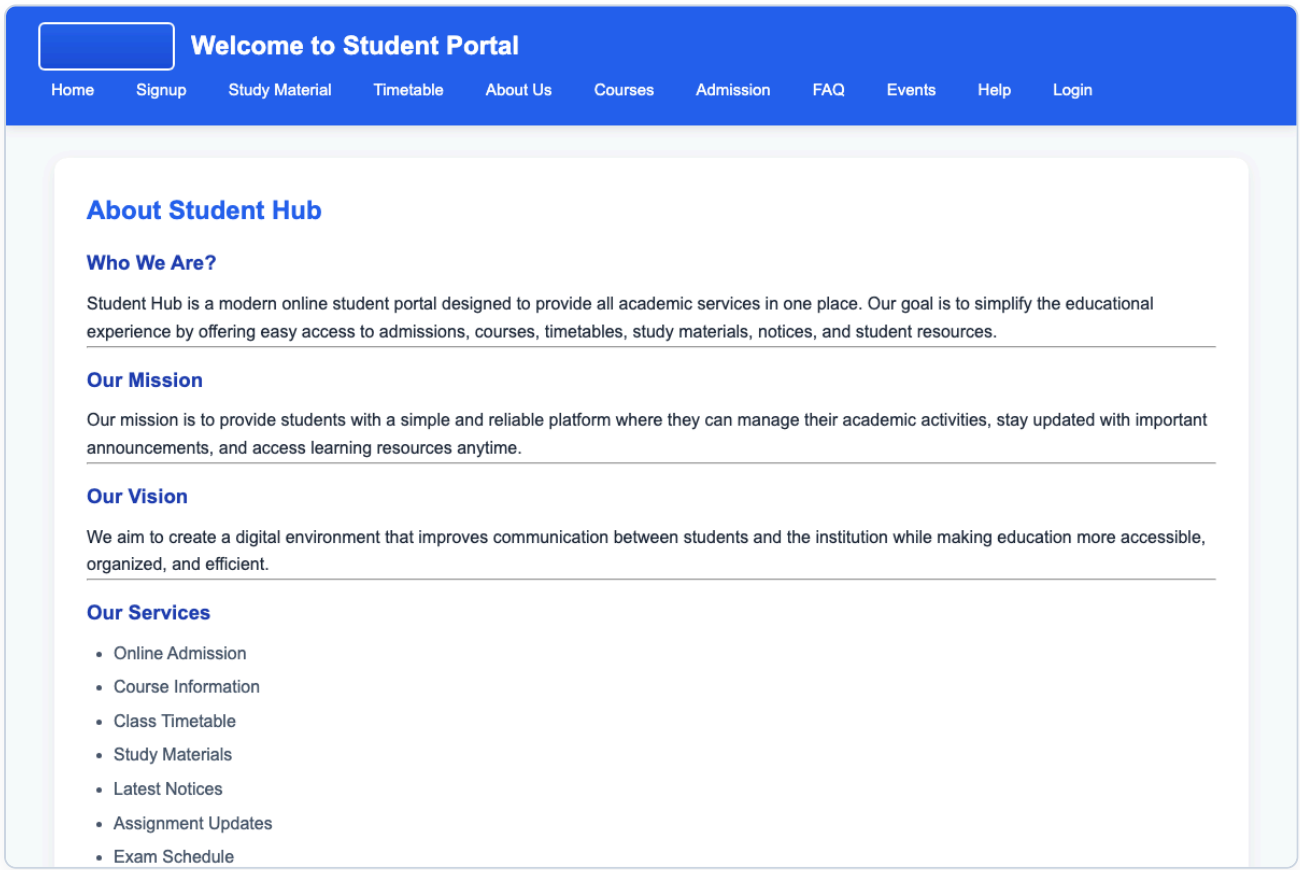
Home Page Structure (Home.html - Excerpt):

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Student Hub - Home</title>
    <link rel="stylesheet" href="style.css">
    <script src="script.js" defer></script>
</head>
<body class="page-home">
    <div class="notification" id="notification">
        <p>New students can register now through the Student Registration Form!</p>
        <button id="closeNotification" aria-label="Close notification">X</button>
    </div>

    <header class="header">
        <div class="logo-section">
            <img src="StudentHub.png" alt="Student Hub Logo">
            <h1>Welcome to Student Portal</h1>
        </div>
        <button class="menu-button" id="menuButton" aria-label="Toggle navigation">≡</button>
        <nav class="navbar" id="navbar">
            <a href="Home.html">Home</a>
            <a href="Signup.html">Signup</a>
            <a href="StudyMaterial.html">Study Material</a>
            <a href="Timetable.html">Timetable<...
```



Home Page Output Screenshot

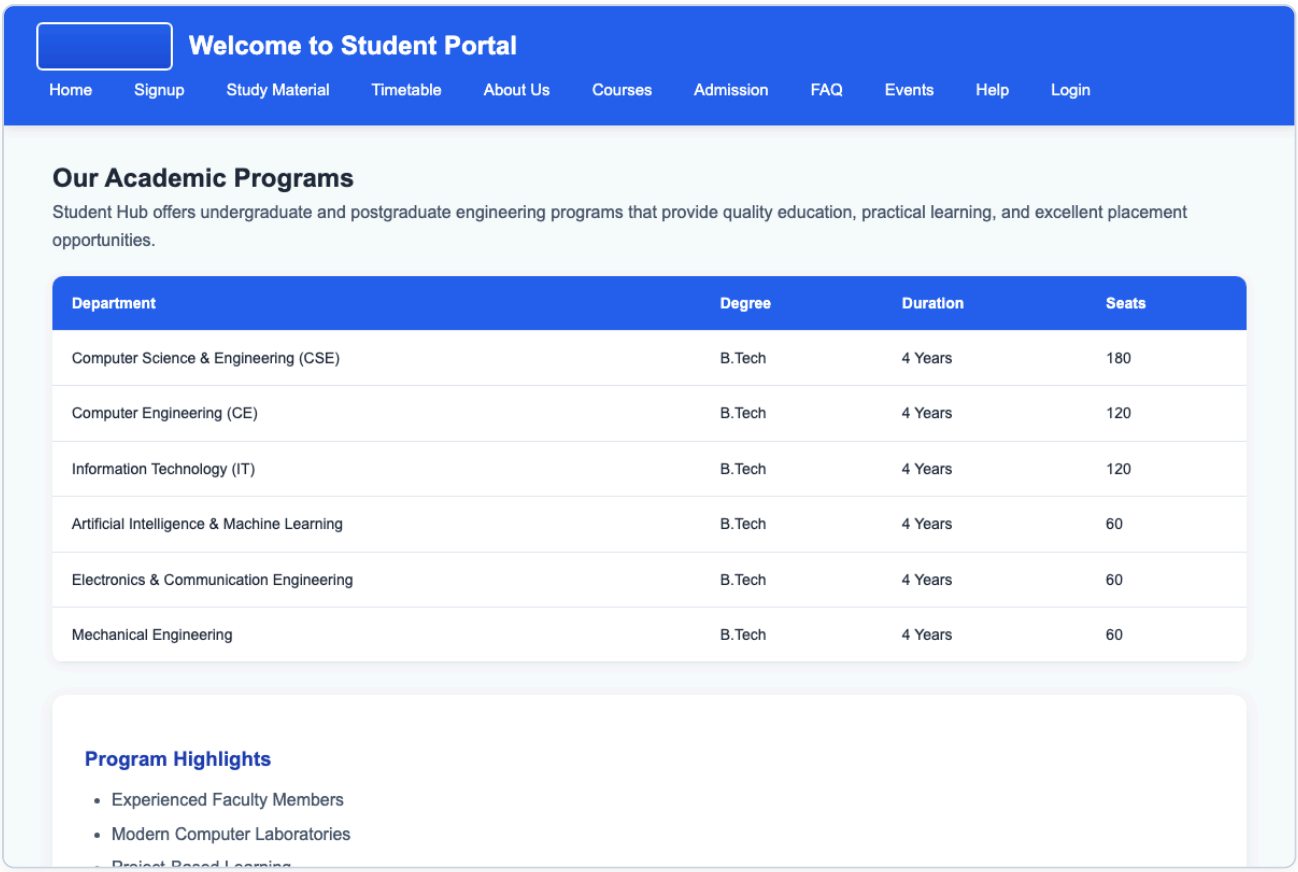


About Us Page Output Screenshot

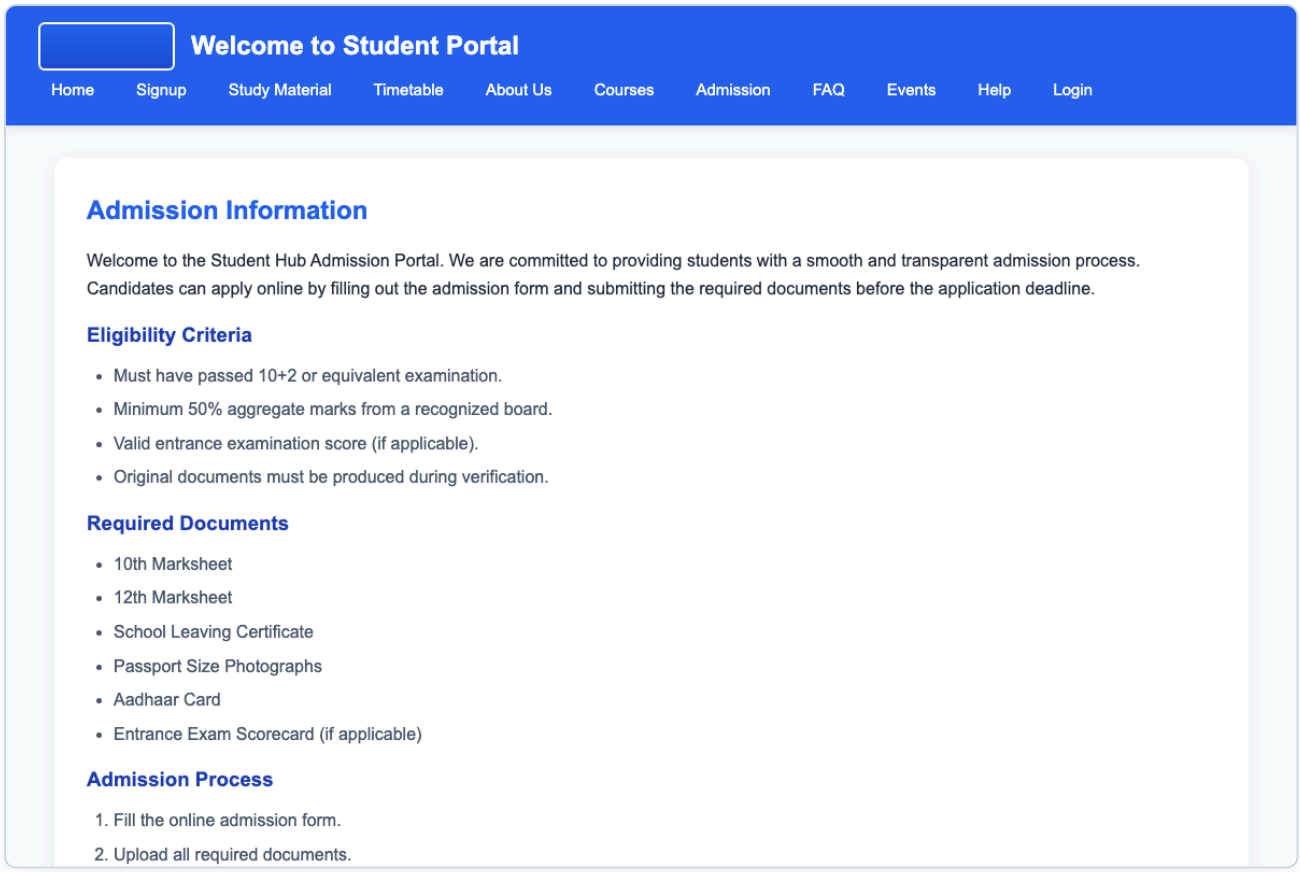
Conclusion:

Created clean, semantic HTML5 page skeletons with accessible tags (<header>, <nav>, <main>, <section>, <footer>) and established inter-page linking across all modules.

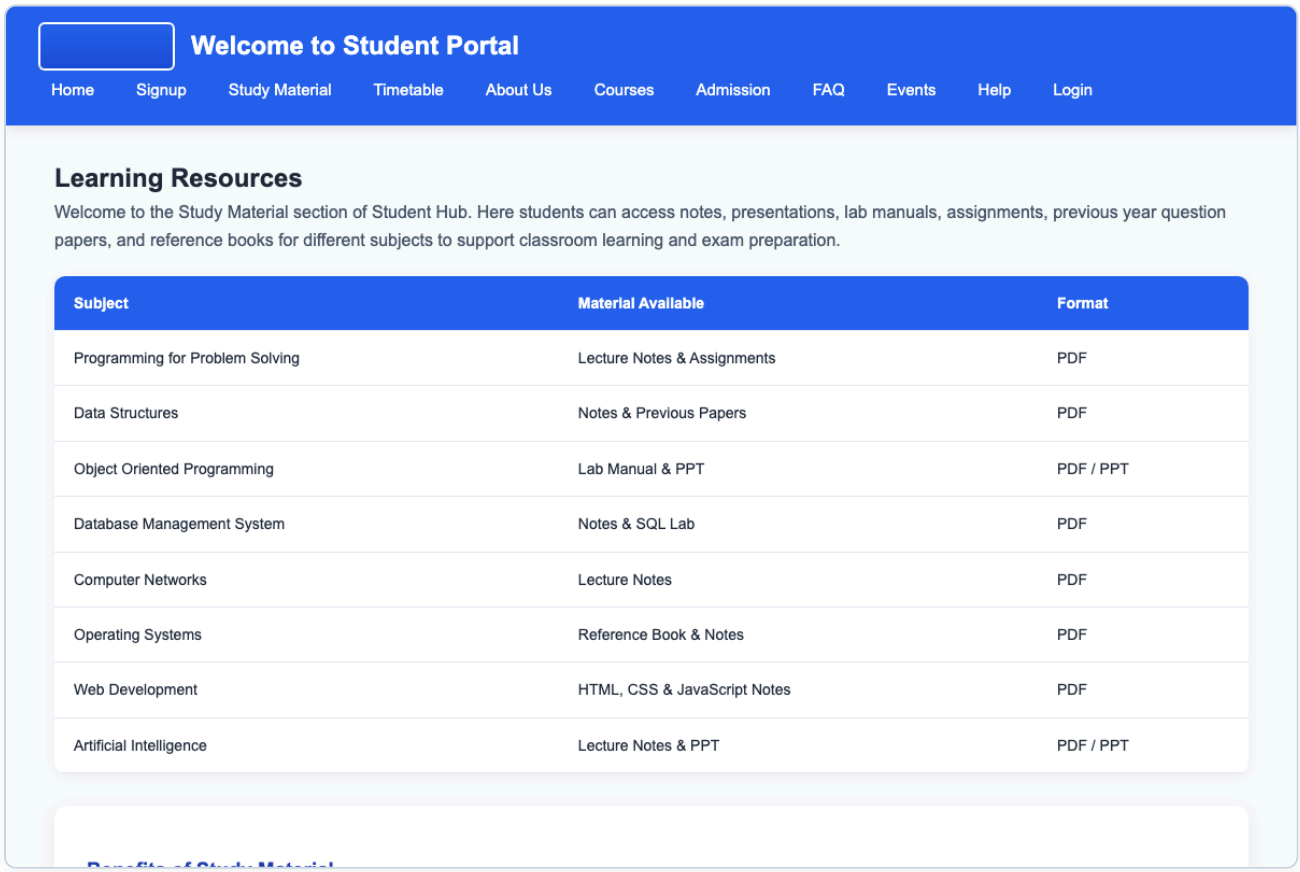
Practical 2: Semantic HTML5 Pages (Continued)



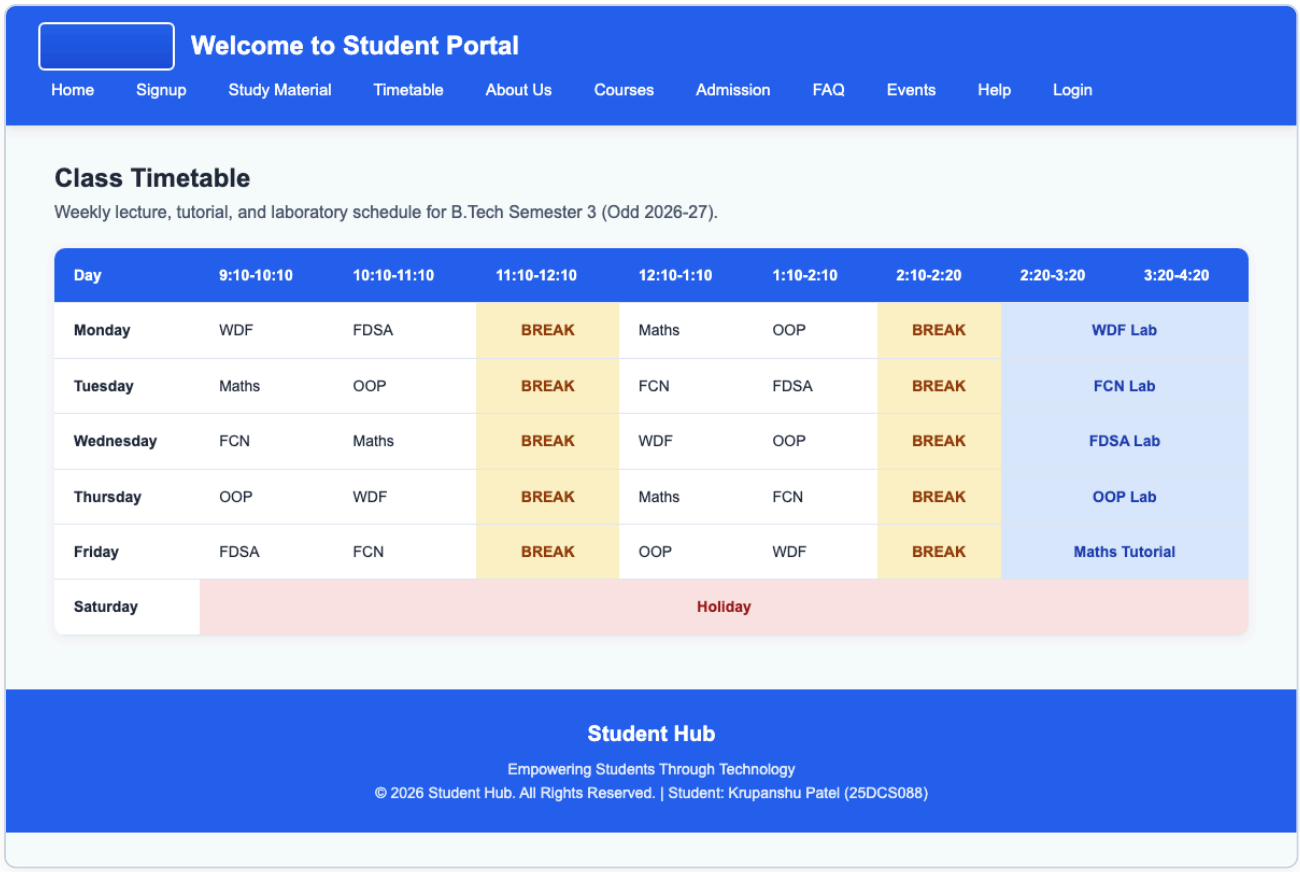
Academic Courses Page Output



Admission Information Page Output



Study Materials & Resources Page Output



Class Timetable Page Output

Practical 3: Responsive UI Design using CSS Grid, Flexbox, and Media Queries

Problem Definition:	Design responsive layouts for Home, About, Registration, Dashboard, and Events pages using CSS Grid and Flexbox. Ensure mobile-first responsiveness across desktop, tablet, and mobile breakpoints.
CO Mapping:	CO2
Key Skills:	CSS3, Media Queries (@media), Flexbox, CSS Grid, Typography, Card layouts

GitHub Code Link: https://github.com/krupanshupatel2347/StudentHub-Portal/blob/main/style.css

Responsive Styling (style.css - Layout Excerpts):

```
/* ============================================================================
   StudentHub Portal – Main Stylesheet
   Subject: Web Development Frameworks (ITUE203)
   Student Name: Krupanshu Patel (25DCS088)
   ============================================================================ */

* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}

body {
    font-family: Arial, sans-serif;
    background-color: #f8fafc;
    color: #1e293b;
    line-height: 1.6;
    transition: background-color 0.3s ease, color 0.3s ease;
}

/* NOTIFICATION BANNER */
.notification {
    background-color: #fff3cd;
    color: #333;
    text-align: center;
    padding: 10px 15px;
    display: flex;
    justify-content: center;
    align-items: center;
    gap: 15px;
    font-size: 15px;
}

#closeNotification {
    background: none;
    border: none;
    font-size: 18px;
    cursor: pointer;
```

```
    font-weight: bold;
    color: #555;
}

#closeNotification:hover {
    color: #000;
}

/* HEADER & NAVBAR */
.header {
    background-color: #2563eb;
    color: white;
    padding: 15px 30px;
    display: flex;
    justify-content: space-between;
    align-items: center;
    flex-wrap: wrap;
    box-shadow: 0 2px 10px rgba(0,0,0,0.1);
}

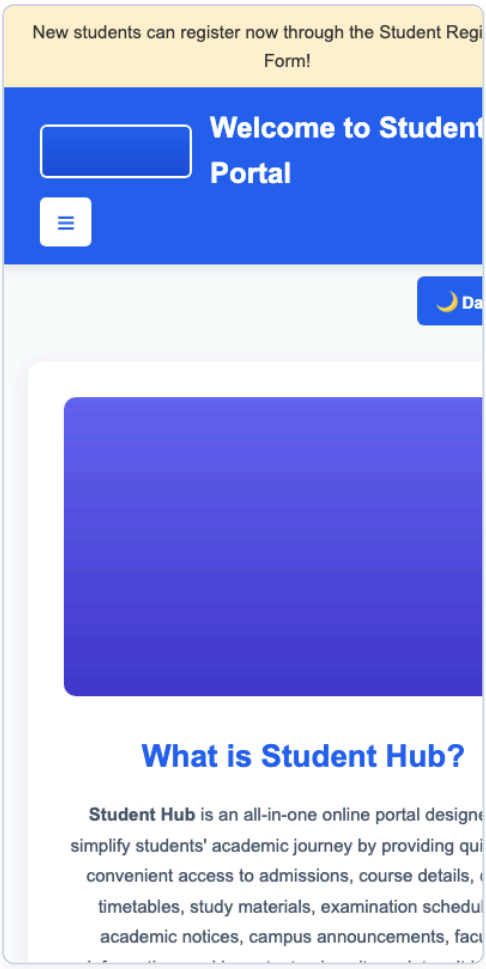
.logo-section {
    display: flex;
    align-items: center;
    gap: 15px;
}

.logo-section img {
    height: 45px;
    border-radius: 6px;
    background: white;
    padding: 2px;
}

.logo-section h1 {
    font-size: 24px;
    margin: 0;
}

.navbar {
    display: flex;
    gap: 15px;
    flex-wrap: wrap;
}

.navbar a {
    color: white;
    text-decoration: n...
```



Mobile Responsive View Output Screenshot

Conclusion:

Implemented clean CSS Flexbox headers and responsive multi-column CSS Grid containers with fluid mobile-first media queries adapting seamlessly across devices.

Practical 4: JavaScript DOM Manipulation, Event Handling, and UI Interactivity

Problem Definition:	Add dynamic UI components such as collapsible FAQ accordion, modal popup dialog, campus image slider, notification banner dismissal, hamburger mobile navigation menu, and light/dark theme switcher using JavaScript.
CO Mapping:	CO3, CO4
Key Skills:	DOM selection & manipulation, Event listeners, classList toggle, LocalStorage persistence

GitHub Code Link: https://github.com/krupanshupatel2347/StudentHub-Portal/blob/main/script.js

DOM Interactivity (script.js - Slider, Modal & Theme Excerpt):

```
/* ============================================================================
   StudentHub Portal – Client-Side Interactivity
   Subject: Web Development Frameworks (ITUE203)
   Student Name: Krupanshu Patel (25DCS088)
   ============================================================================ */

document.addEventListener("DOMContentLoaded", function () {
    // 1. NOTIFICATION BANNER CLOSE
    const notification = document.getElementById("notification");
    const closeNotification = document.getElementById("closeNotification");
    if (notification && closeNotification) {
        closeNotification.addEventListener("click", function () {
            notification.style.display = "none";
        });
    }

    // 2. MOBILE HAMBURGER MENU
    const menuButton = document.getElementById("menuButton");
    const navbar = document.getElementById("navbar");
    if (menuButton && navbar) {
        menuButton.addEventListener("click", function () {
            navbar.classList.toggle("show-menu");
        });
    }

    // 3. LIGHT / DARK THEME SWITCHER WITH LOCALSTORAGE
    const themeButton = document.getElementById("themeButton");
    const savedTheme = localStorage.getItem("studenthub_theme");
    if (savedTheme === "dark") {
        document.body.classList.add("dark-mode");
        if (themeButton) themeButton.textContent = "☀️ Light Mode";
    }

    if (themeButton) {
        themeButton.addEventListener("click&qu...
```

New students can register now through the Student Registration Form!

X

Student Hub Logo

Welcome to Student Portal

≡

Home Signup Study Material Timetable About Us Courses Admission FAQ Events Help Login

Dark Mode

College Campus

What is Student Hub?

Student Hub is an all-in-one online portal designed to simplify students' academic journey by providing quick and convenient access to admissions, course details, class timetables, study materials, examination schedules, academic notices, campus announcements, faculty information, and important university updates. It brings every essential academic service together in one place, making it easier for students to stay organized, informed, and connected throughout their educational journey.

Whether you are a new student applying for admission or an existing student managing your daily academic activities, Student Hub offers a simple, secure, and user-friendly experience. Students can explore available courses, check class schedules, download study resources, receive important notifications, access academic information, and stay updated with campus events from anywhere at any time. Our mission is to create a smart digital learning environment that improves communication, enhances productivity, and supports students in achieving their academic goals with ease.

View Course Information

CHARUSAT

CHARUSAT

Welcome to CHARUSAT campus.

Previous Next

×

Courses

Student Hub provides information about different courses offered by the university.

Students can explore course details, duration, eligibility, and academic information.

Dark Mode UI Theme Output Screenshot

New students can register now through the Student Registration Form!

X

Student Hub Logo

Welcome to Student Portal

≡

Home Signup Study Material Timetable About Us Courses Admission FAQ Events Help Login

Dark Mode

College Campus

What is Student Hub?

Student Hub is an all-in-one online portal designed to simplify students' academic journey by providing quick and convenient access to admissions, course details, class timetables, study materials, examination schedules, academic notices, campus announcements, faculty information, and important university updates. It brings every essential academic service together in one place, making it easier for students to stay organized, informed, and connected throughout their educational journey.

Whether you are a new student applying for admission or an existing student managing your daily academic activities, Student Hub offers a simple, secure, and user-friendly experience. Students can explore available courses, check class schedules, download study resources, receive important notifications, access academic information, and stay updated with campus events from anywhere at any time. Our mission is to create a smart digital learning environment that improves communication, enhances productivity, and supports students in achieving their academic goals with ease.

View Course Information

CHARUSAT

CHARUSAT

Welcome to CHARUSAT campus.

Previous Next

×

Courses

Student Hub provides information about different courses offered by the university.

Students can explore course details, duration, eligibility, and academic information.

Course Information Modal Dialog Output

Conclusion:

Implemented responsive client-side event handlers including theme toggle with localStorage persistence, dynamic modal dialogs, and automated campus image carousels.

Practical 5: Registration Form with Frontend Validation and User-Friendly Error Handling

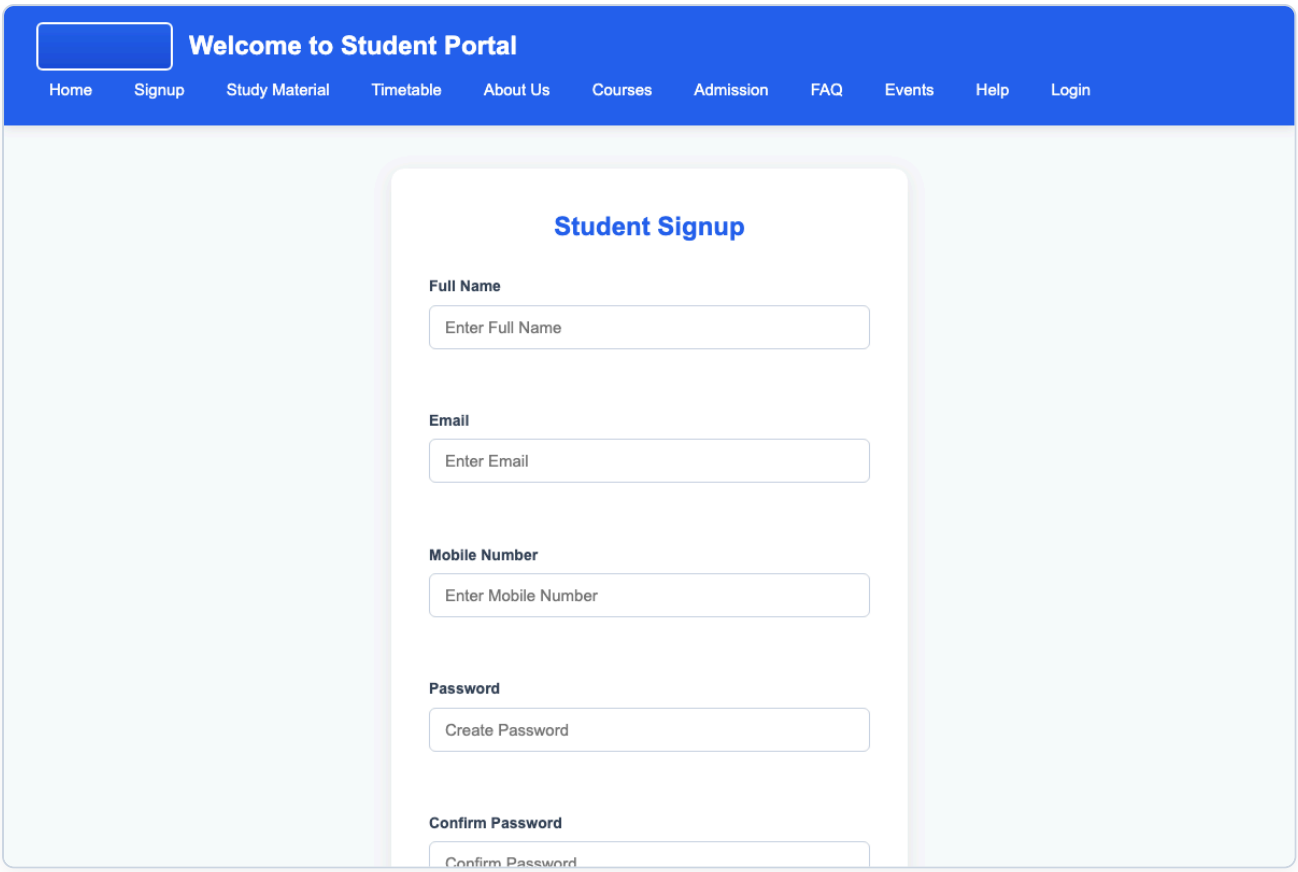
Problem Definition:	Create a student registration form with HTML5 input types and JavaScript validation for name, email, mobile number, password, confirm password, course, year, gender, and terms acceptance. Use Regular Expressions for validations.
CO Mapping:	CO1, CO3
Key Skills:	HTML forms, JS Regex validation, realtime error messages, input sanitization

GitHub Code Link: https://github.com/krupanshupatel2347/StudentHub-Portal/blob/main/Signup.html

Frontend Regex Validation Logic (script.js):

```
const nameRegex = /^[A-Za-z ]{2,50}$/;
const emailRegex = /^[\w.-]+@[\w.-]+\.\w{2,}$/;
const mobileRegex = /^[6-9]\d{9}$/;
const passwordRegex = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d).{8,}$/;

// Validate on submit
if (!nameRegex.test(name)) {
    document.getElementById("nameError").textContent = "Enter a valid name (letters & spaces onl
y).";
    valid = false;
}
if (!emailRegex.test(email)) {
    document.getElementById("emailError").textContent = "Enter a valid email address.";
    valid = false;
}
if (!mobileRegex.test(mobile)) {
    document.getElementById("mobileError").textContent = "Enter a valid 10-digit mobile number.";
    valid = false;
}
if (!passwordRegex.test(password)) {
    document.getElementById("passwordError").textContent = "Password must be 8+ chars with upperca
se, lowercase, and number.";
    valid = false;
}
```



Registration Form UI Output Screenshot

Conclusion:

Designed a comprehensive client-side form validator preventing malformed submissions, verifying email formats, enforcing password strength rules, and displaying inline error messages.

Practical 6: Rendering External JSON Data using Fetch API, Search and Filter

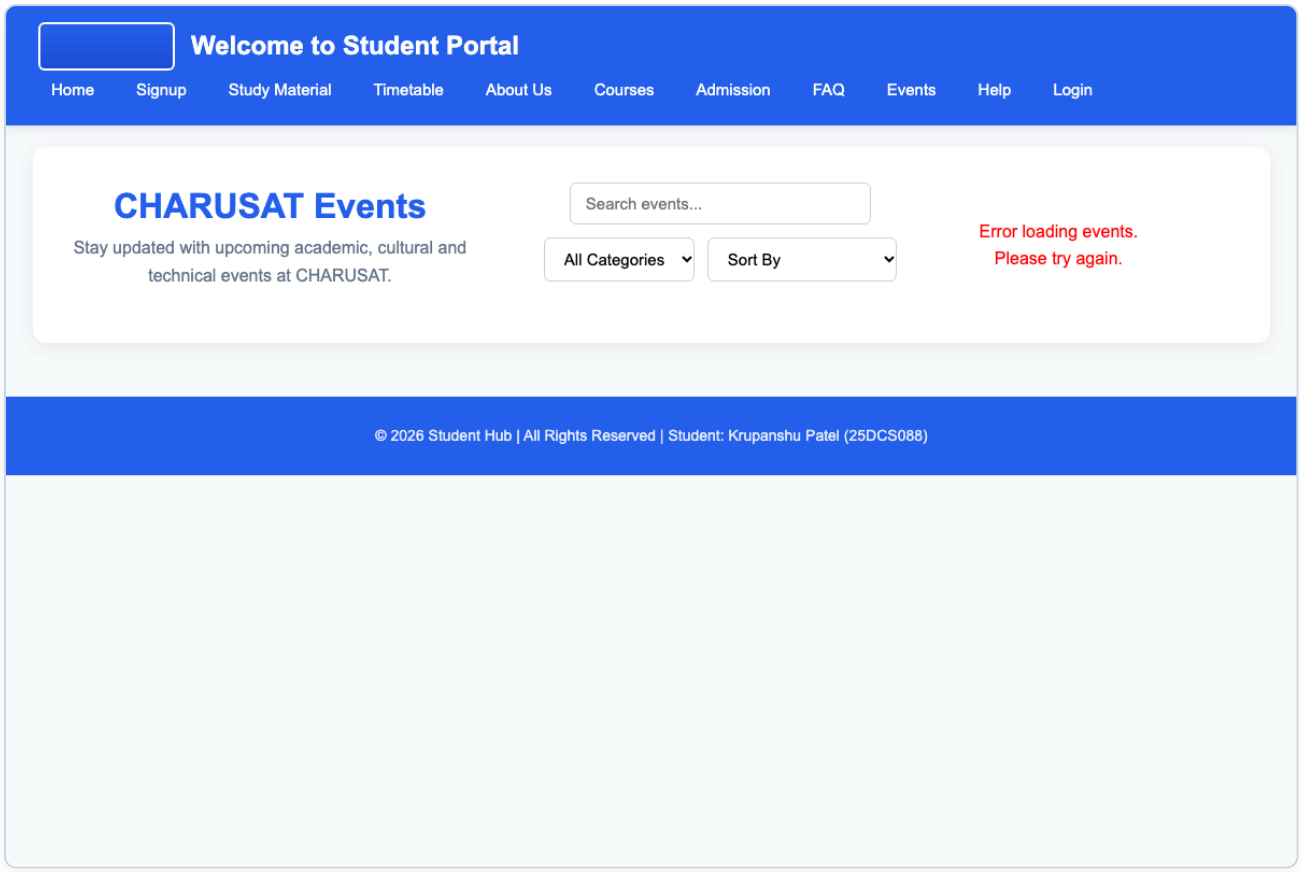
Problem Definition:	Display event lists, student profiles, notices, or FAQs from external JSON files using Fetch API. Implement dynamic rendering, search, category filter, sorting, and pagination.
CO Mapping:	CO3, CO4
Key Skills:	JSON parsing, Fetch API, Array map/filter/sort, client-side pagination

GitHub Code Link: https://github.com/krupanshupatel2347/StudentHub-Portal/blob/main/event.html

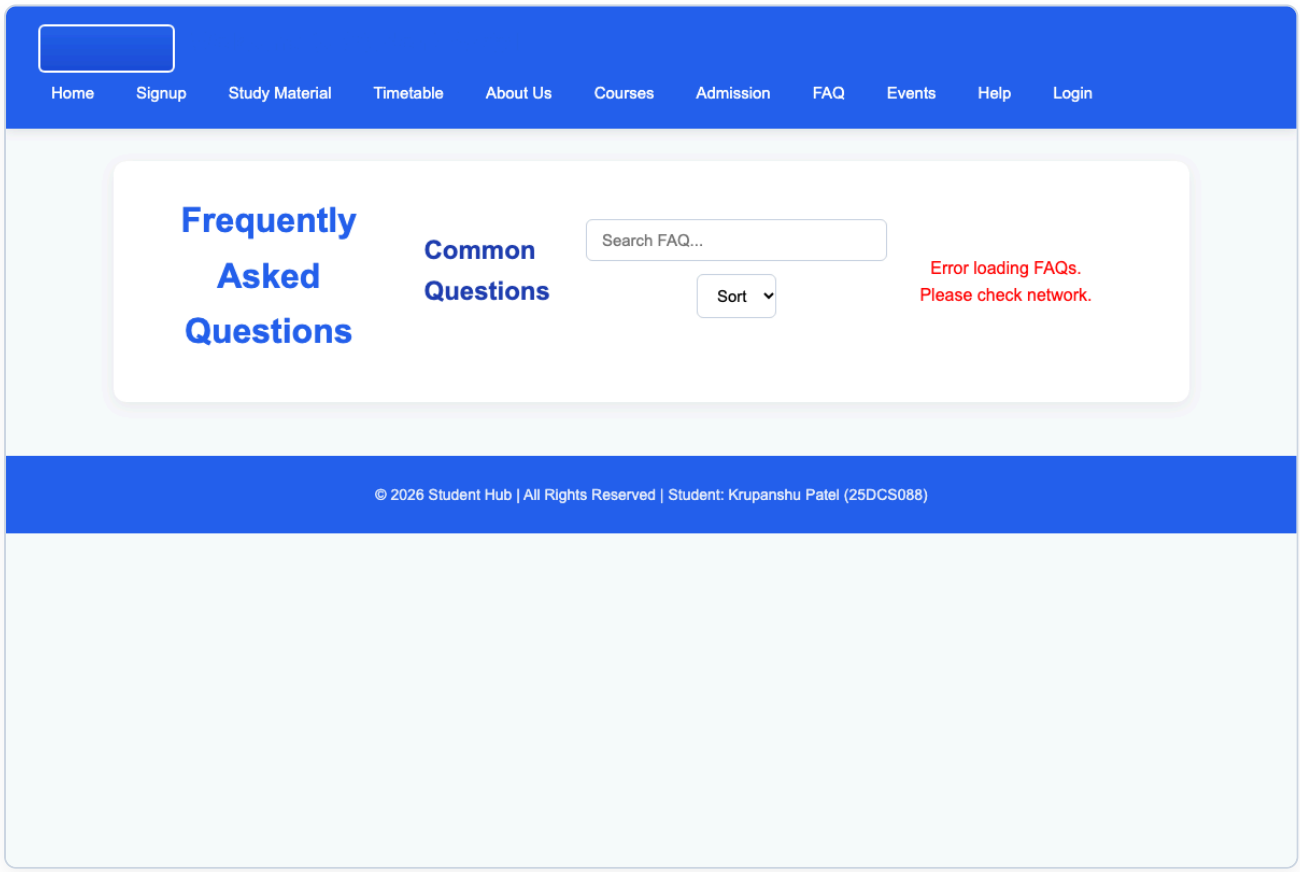
Fetch API & Filter Logic (script.js - Excerpt):

```
fetch("events.json")
    .then(res => res.json())
    .then(data => {
        eventData = data;
        filteredEvents = [...eventData];
        displayEvents();
    });

function filterAndSortEvents() {
    const query = eventSearch.value.toLowerCase().trim();
    const cat = eventCategory.value;
    filteredEvents = eventData.filter(ev => {
        const matchesSearch = ev.name.toLowerCase().includes(query) || ev.venue.toLowerCase().incl
udes(query);
        const matchesCat = (cat === "all") || (ev.category === cat);
        return matchesSearch && matchesCat;
    });
    displayEvents();
}
```



Dynamic Events Catalog Output (events.json)



Dynamic FAQ Search & Pagination Output (faqs.json)

Conclusion:

Learned how to decouple frontend presentation from backend data storage by consuming JSON endpoints via asynchronous Fetch API, filtering in memory, and implementing pagination.

Practical 7: PHP Form Processing with Server-Side Validation and CSV/JSON File Storage

Problem Definition:	Process submitted registration/contact form data using PHP. Validate and sanitize inputs on the server side and store records in CSV or JSON file format. Display success/error messages.
CO Mapping:	CO1, CO5
Key Skills:	PHP POST handling, Server-side sanitization, File locking (flock), CSV storage

GitHub Code Link: https://github.com/krupanshupatel2347/StudentHub-Portal/blob/main/signup.php

PHP Server-Side Processing (signup.php - Excerpt):

```
<?php
/**
 * Practical 9: Secure User Registration with Database Insert, Duplicate Email Check, and Password Hashing
 * Student: Krupanshu Patel (25DCS088)
 */
mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
require_once "db.php";

if ($_SERVER["REQUEST_METHOD"] !== "POST") {
    header("Location: Signup.html");
    exit();
}

$fullName = trim($_POST["fullName"] ?? "");
$email = trim($_POST["email"] ?? "");
$mobile = trim($_POST["mobile"] ?? "");
$password = $_POST["password"] ?? "";
$confirmPassword = $_POST["confirmPassword"] ?? "";
$course = trim($_POST["course"] ?? "");
$year = trim($_POST["year"] ?? "");
$gender = trim($_POST["gender"] ?? "");
$terms = $_POST["terms"] ?? "";

$errors = [];

/* Full Name Validation */
if ($fullName === "") {
    $errors[] = "Full name is required.";
} elseif (!preg_match("/^[a-zA-Z ]+$/", $fullName)) {
    $errors[] = "Full name can contain only letters and spaces.";
}

/* Email Validation */
if ($email === "") {
    $errors[] = "Email is required.";
} elseif (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
    $errors[] = "Enter a valid email address.";
```

```
}

/* Mobile Validation */
if ($mobile === "") {
    $errors[] = "Mobile n...
```

Stored CSV Record (data/users.csv):

```
"Full Name","Email","Mobile","Password","Course","Year","Gender"
"Krupanshu Patel","krupanshupatel2347@gmail.com","9876543210","$2y$10$92IXUNpkjO0rOQ5byMi.Ye4oKoEa
3Ro9llC/.og/at2.uheWG/igi","B.Tech CSE","2","Male"
```

Conclusion:

Implemented server-side validation using filter_var and regex, hashed sensitive passwords, and safely persisted user submissions into CSV format with file concurrency locking.

Practical 8: MySQL Schema Design, ER Model, PDO Connectivity, and Prepared Statements

Problem Definition:	Design the database for StudentHub with students, events, and registrations. Create PHP database connection using MySQL statements and PDO prepared statements.
CO Mapping:	CO1, CO5
Key Skills:	Database normalization, ER modeling, PDO error modes, MySQLi, fetch queries

GitHub Code Link: https://github.com/krupanshupatel2347/StudentHub-Portal/blob/main/db.php

Database Connection Script (db.php):

```php
<?php
/**
 * Database Connection (db.php)
 * Student: Krupanshu Patel (25DCS088)
 */
$host = "localhost";
$dbname = "studenthub_db";
$username = "root";
$password = "";

// PDO Connection
try {
    $pdo = new PDO(
        "mysql:host=$host;dbname=$dbname;charset=utf8mb4",
        $username,
        $password
    );
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    $pdo->setAttribute(PDO::ATTR_DEFAULT_FETCH_MODE, PDO::FETCH_ASSOC);
} catch (PDOException $error) {
    error_log($error->getMessage());
    // Fallback or warning if MySQL server is offline during static checks
}

// MySQLi Connection
$conn = @new mysqli($host, $username, $password, $dbname);
if ($conn && !$conn->connect_error) {
    $conn->set_charset("utf8mb4");
}
?>
```

Fetch Test Script (fetch.php):

```php
<?php
/**
 * Database connectivity verification (fetch.php)
 * Student: Krupanshu Patel (25DCS088)
 */
```

```php
require_once "db.php";

if (!isset($pdo)) {
    die("Database connection failed or not established.");
}

try {
    $sql = "SELECT student_id, full_name, email, mobile FROM students";
    $stmt = $pdo->prepare($sql);
    $stmt->execute();
    $students = $stmt->fetchAll();

    echo "<h2>Student Details</h2>";
    foreach ($students as $student) {
        echo "ID: " . htmlspecialchars((string)$student["student_id"]) . "<br>";
        echo "Name: " . htmlspecialchars($student["full_name"]) . "<br>";
        echo "Email: " . htmlspecialchars($student["email"]) . "<br>";
        echo "Mobile: " . htmlspecialchars($student["mobile"]) . "<br><hr>";
    }
} catch (Exception $e) {
    echo "Error fetching records: " . htmlspecialchars($e->getMessage());
}
?>
```

Conclusion:

Configured standardized PDO and MySQLi connectivity handlers with UTF-8 character encoding, exception handling, and verified database table reads.

Practical 9: Secure User Registration with Database Insert, Duplicate Email Check, and Password Hashing

Problem Definition:	Implement secure user registration by inserting user details into MySQL using MySQLi. Validate input on frontend and backend, check duplicate email/username, hash passwords using password_hash(), and display appropriate messages.
CO Mapping:	CO1, CO3, CO5
Key Skills:	MySQLi prepared statements, Duplicate key collision check, Bcrypt password hashing

GitHub Code Link: https://github.com/krupanshupatel2347/StudentHub-Portal/blob/main/signup.php

Duplicate Email Check & Secure Insert (signup.php):

```
/* Duplicate Email Check */
$checkSql = "SELECT user_id FROM users WHERE email = ?";
$checkStmt = $conn->prepare($checkSql);
$checkStmt->bind_param("s", $email);
$checkStmt->execute();
$checkStmt->store_result();

if ($checkStmt->num_rows > 0) {
    exit("This email is already registered. Please use another email.");
}
$checkStmt->close();

/* Hash Password */
$passwordHash = password_hash($password, PASSWORD_DEFAULT);

/* Insert User */
$insertSql = "INSERT INTO users (full_name, email, mobile, password, course, year, gender) VALUES
(?, ?, ?, ?, ?, ?, ?)";
$insertStmt = $conn->prepare($insertSql);
$insertStmt->bind_param("ssssssis", $fullName, $email, $mobile, $passwordHash, $course, $year, $gen
der);
$insertStmt->execute();
```

Conclusion:

Secured user account registration by mitigating SQL injection vulnerabilities with prepared statements, preventing duplicate registrations, and storing passwords via one-way Bcrypt hashing.

Practical 10: Secure Login/Logout with Sessions, Role-Based Access, Timeout, and Remember-Me Token

Problem Definition:	Develop secure login and logout functionality using PHP sessions. Verify password using password_verify(), regenerate session ID after login, restrict dashboard access, implement role-based redirection, session timeout, and secure remember-me token.
CO Mapping:	CO1, CO5
Key Skills:	PHP Sessions, RBAC guards, Session timeout, Dual-token remember-me authentication

GitHub Code Link: https://github.com/krupanshupatel2347/StudentHub-Portal/blob/main/process.php

Session RBAC Redirection (process.php - Excerpt):

```
<?php
/**
 * Practical 10: Secure Login Processing with Password Verification and RBAC Redirection
 * Student: Krupanshu Patel (25DCS088)
 */
if (session_status() === PHP_SESSION_NONE) {
    ini_set("session.use_strict_mode", "1");
    session_start();
}

require_once "db.php";
require_once "remember.php";

if ($_SERVER["REQUEST_METHOD"] !== "POST") {
    header("Location: Login.html");
    exit();
}

$email = trim($_POST["email"] ?? "");
$password = $_POST["password"] ?? "";
$selectedRole = $_POST["role"] ?? "student";

if ($email === "" || $password === "") {
    header("Location: Login.html?error=empty");
    exit();
}

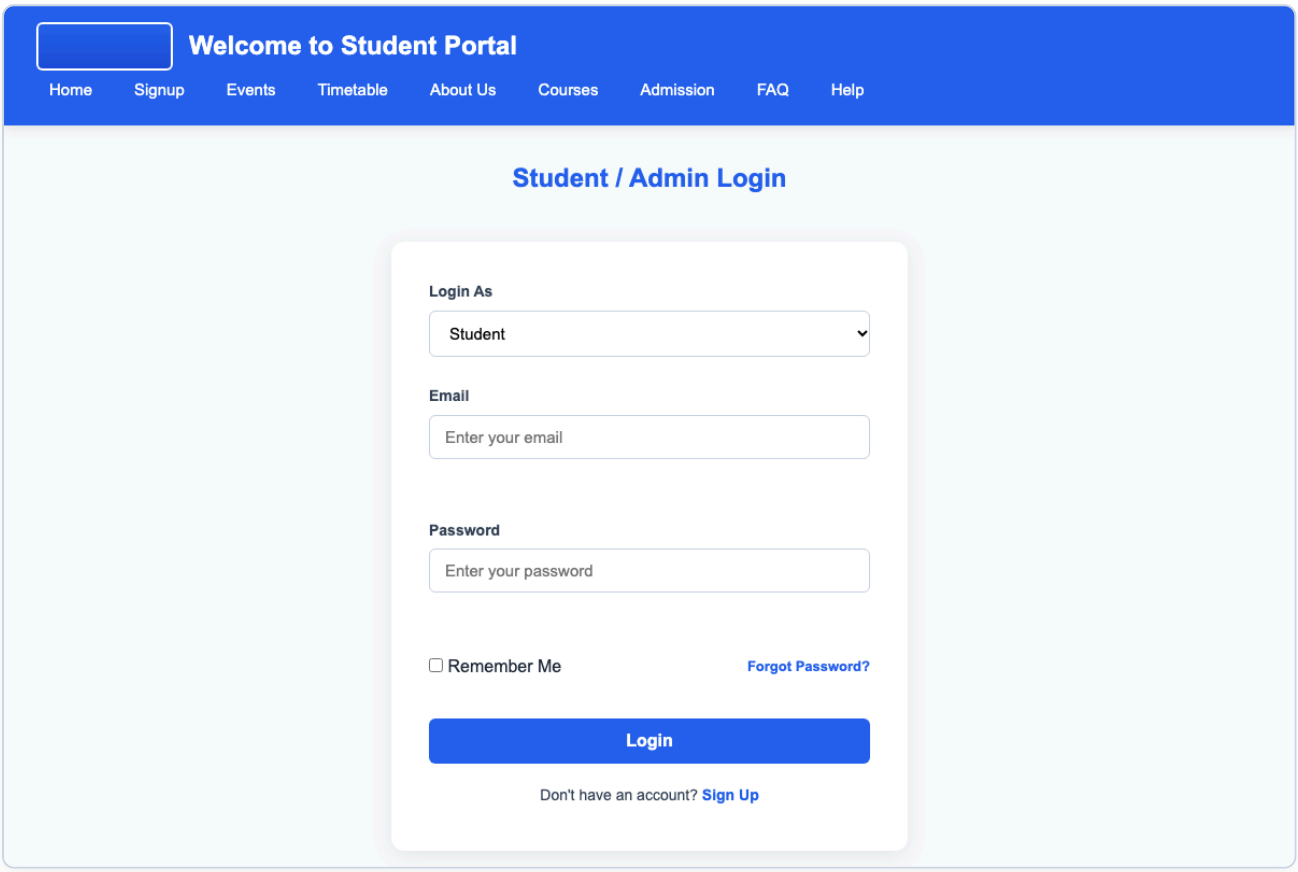
if (!filter_var($email, FILTER_VALIDATE_EMAIL) || !in_array($selectedRole, ["student", "admin"], t
rue)) {
    header("Location: Login.html?error=invalid");
    exit();
}

$stmt = $conn->prepare(
    "SELECT u.user_id, u.full_name, u.email, u.password, COALESCE(ur.role, 'student') AS role
     FROM users u
     LEFT JOIN user_roles ur ON ur.user_id = u.user_id
     WHERE u.email = ?"
```

```
);

if (!$stmt) {
    error_log($conn->error);
    http_response_code(500);
    exit("Unable to process login right now.");
}

$stmt->bind_param("s", $email);
$stmt->execute();
$result = $stmt->get_result();
$user = $result->fetch_a...
```



Student / Admin Login Interface Output

Conclusion:

Constructed a robust authentication subsystem with session fixation protection via session_regenerate_id(), 15-minute inactivity timeouts, role validation, and secure cryptographic remember-me tokens.

Practical 11: Student Management CRUD with Search and Filter

Problem Definition:	Create a student management module to add, view, edit, delete, search and filter student records. All database operations must use MySQLi statements and clear success/failure messages.
CO Mapping:	CO1, CO4, CO5
Key Skills:	PHP-MySQL CRUD operations, dynamic search, multi-field filters, UI feedback

GitHub Code Link: https://github.com/krupanshupatel2347/StudentHub-Portal/blob/main/students.php

Student CRUD Implementation (students.php - Excerpt):

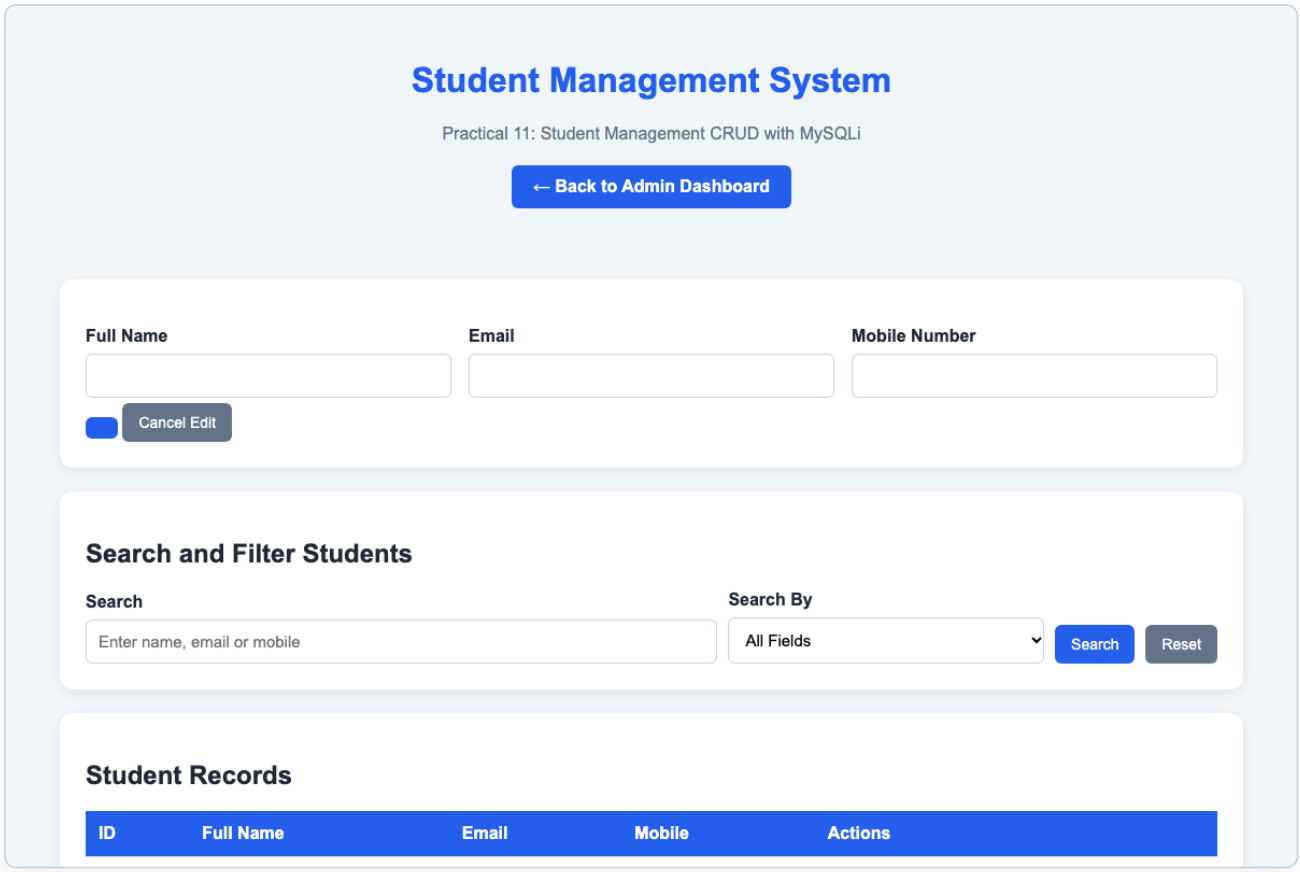
```
<?php
/**
 * Practical 11: Student Management CRUD with Search and Filter
 * Student: Krupanshu Patel (25DCS088)
 */
require_once "db.php";

$message = "";
$messageType = "";
$editStudent = null;

if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $action = $_POST["action"] ?? "";

    if ($action === "add" || $action === "update") {
        $fullName = trim($_POST["full_name"] ?? "");
        $email = trim($_POST["email"] ?? "");
        $mobile = trim($_POST["mobile"] ?? "");
        $studentId = (int)($_POST["student_id"] ?? 0);

        if ($fullName === "" || $email === "" || $mobile === "") {
            $message = "Please fill in all fields.";
            $messageType = "error";
        } elseif (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
            $message = "Please enter a valid email address.";
            $messageType = "error";
        } elseif (!preg_match('/^[0-9]{10}$/', $mobile)) {
            $message = "Mobile number must contain exactly 10 digits.";
            $messageType = "error";
        } else {
            if ($action === "add") {
                $stmt = $conn->prepare("INSERT INTO students (full_name, email, mobile) VALUES (?,
?, ?)");
                if ($stmt) {
 ...
```



Student Management CRUD Interface Output

Conclusion:

Built a full-featured administrative data-management table supporting adding new students, in-place editing, confirmed deletion, and flexible multi-criteria search filters.

Practical 12: Event Management CRUD with Poster Upload and File Validation

Problem Definition:	Develop event management functionality for admin users. Admin can add, view, update, delete, and upload event posters with file type and file size validation.
CO Mapping:	CO1, CO4, CO5
Key Skills:	File uploads (enctype), MIME validation (finfo), file size restriction, safe filename generation

GitHub Code Link: https://github.com/krupanshupatel2347/StudentHub-Portal/blob/main/events.php

Event Poster Upload Validation (events.php - Excerpt):

```php
<?php
/**
 * Practical 12: Event Management CRUD with Poster Upload and File Validation
 * Student: Krupanshu Patel (25DCS088)
 */
error_reporting(E_ALL);
ini_set('display_errors', '1');
require_once "db.php";

$message = "";
$messageType = "";
$uploadDirectory = __DIR__ . "/uploads/";

if (!is_dir($uploadDirectory)) {
    mkdir($uploadDirectory, 0755, true);
}

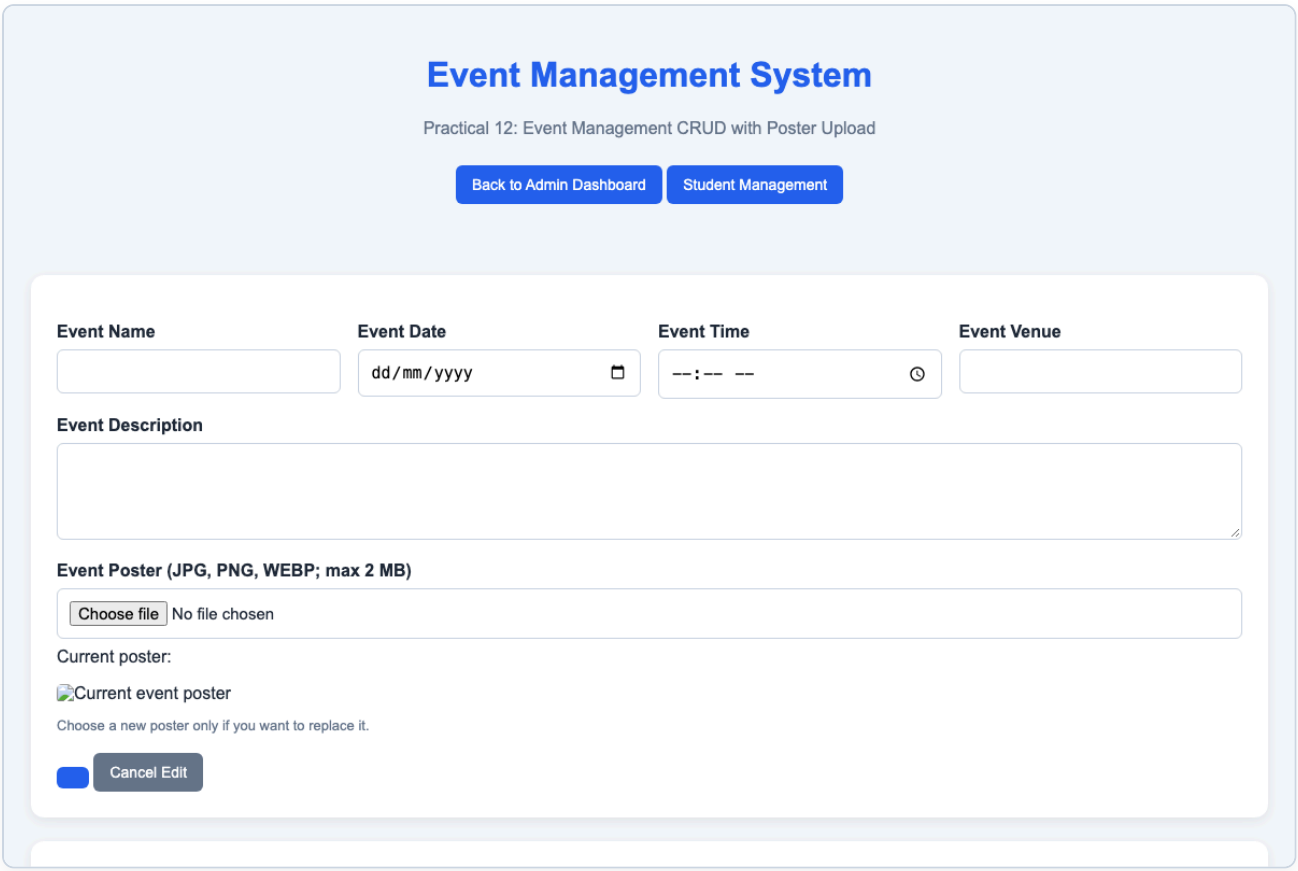
function clean($value)
{
    return htmlspecialchars((string)$value, ENT_QUOTES, "UTF-8");
}

if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $action = $_POST["action"] ?? "";

    if ($action === "add" || $action === "update") {
        $eventId = (int)($_POST["event_id"] ?? 0);
        $eventName = trim($_POST["event_name"] ?? "");
        $eventDate = $_POST["event_date"] ?? "";
        $eventTime = $_POST["event_time"] ?? "";
        $eventVenue = trim($_POST["event_venue"] ?? "");
        $eventDescription = trim($_POST["event_description"] ?? "");
        $posterName = null;
        $oldPoster = "";

        if ($action === "update" && $eventId > 0) {
            $find = $conn->prepare("SELECT poster FROM events WHERE event_id = ?");
            $find->bind_param("i", $eventId);
```

```
            $find->execute();
    ...
```



Event Management CRUD with Poster Upload Output

Conclusion:

Implemented secure multi-part form processing for university events, enforcing a 2MB upload limit, validating genuine image MIME types via PHP finfo, and cleaning up obsolete image assets upon update or deletion.

Practical 13: PHP JSON API and AJAX/Fetch-Based CRUD without Page Reload

Problem Definition:	Create simple PHP JSON API endpoints for events or students and consume them using Fetch API. Perform list, add, update, and delete operations asynchronously without full page reload.
CO Mapping:	CO3, CO4, CO5
Key Skills:	RESTful JSON API, HTTP methods (GET, POST, PUT, DELETE), Asynchronous fetch, DOM updating

GitHub Code Link: https://github.com/krupanshupatel2347/StudentHub-Portal/blob/main/api.php

RESTful PHP JSON API (api.php - Excerpt):

```
<?php
/**
 * Practical 13: RESTful PHP JSON API for Event Management
 * Student: Krupanshu Patel (25DCS088)
 */
header("Content-Type: application/json; charset=UTF-8");
require_once "db.php";

function sendResponse($statusCode, $response)
{
    http_response_code($statusCode);
    echo json_encode($response);
    exit;
}

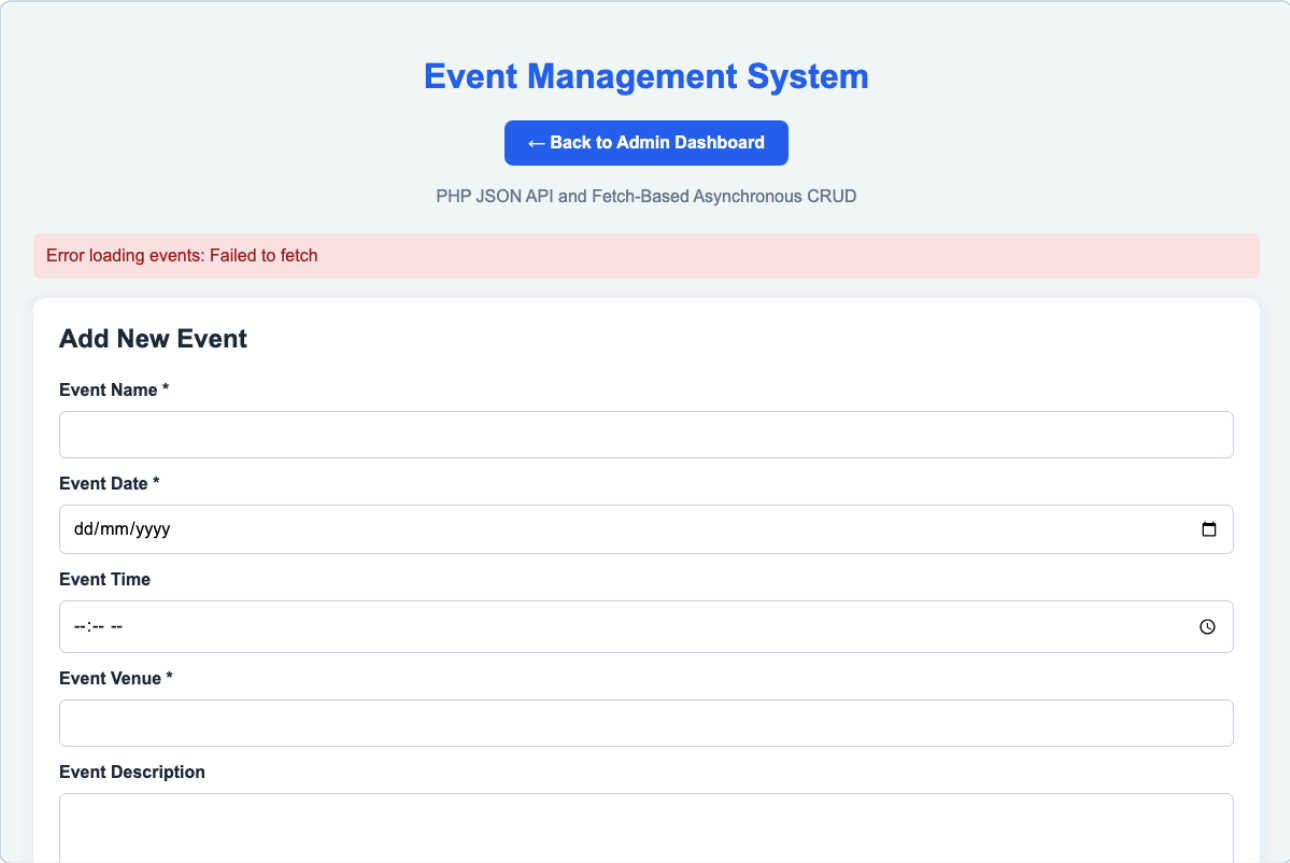
function getInputData()
{
    $input = file_get_contents("php://input");
    $data = json_decode($input, true);
    if (!is_array($data)) {
        sendResponse(400, [
            "success" => false,
            "message" => "Invalid JSON data."
        ]);
    }
    return $data;
}

function validateEvent($data)
{
    $eventName = trim($data["event_name"] ?? "");
    $eventDate = trim($data["event_date"] ?? "");
    $eventVenue = trim($data["event_venue"] ?? $data["venue"] ?? "");

    if ($eventName === "" || $eventDate === "" || $eventVenue === "") {
        sendResponse(400, [
            "success" => false,
```

```
            "message" => "Event name, date and venue are required."
        ]);
    }

    $dateObject = DateTime::createFromFormat("!Y-m-d", $eventDate);
    if (!$dateObject || $dateObject->format("Y-m-d") !== $eventDate) {
        sendResponse(400, [
            "success" => false,
            "message&quo...
```



AJAX Asynchronous Event Management Interface Output (index.html)

Conclusion:

Developed a RESTful JSON API adhering to standard HTTP verbs and constructed an asynchronous single-page interface with real-time UI updates without browser reloads.

Practical 14: Admin Dashboard with Role-Based Access Control, Dynamic Menu, and Audit Log

Problem Definition:	Develop an admin dashboard that displays user statistics, student count, event count and latest registrations. Restrict access to admin users only and show dynamic menu options based on role.
CO Mapping:	CO1, CO4, CO5
Key Skills:	SQL aggregation, RBAC dynamic menus, Audit log tracking, Interactive profile modal

GitHub Code Link: https://github.com/krupanshupatel2347/StudentHub-Portal/blob/main/admin_dashboard.php

Admin Dashboard Metrics & Audit Logging (admin_dashboard.php - Excerpt):

```php
<?php
/**
 * Practical 14: Admin Dashboard with Role-Based Access Control, Dynamic Menu, and Audit Log
 * Student: Krupanshu Patel (25DCS088)
 */
require_once "auth.php";
requireRole("admin");
require_once "db.php";

mysqli_report(MYSQLI_REPORT_OFF);

$fullName = $_SESSION["full_name"] ?? "Admin";
$userId = (int)($_SESSION["user_id"] ?? 0);
$userRole = $_SESSION["role"] ?? "student";

function e($value) {
    return htmlspecialchars((string)($value ?? ""), ENT_QUOTES, "UTF-8");
}

$students = [];
$latestRegistrations = [];
$auditLogs = [];
$error = "";
$totalAccounts = 0;
$totalStudents = 0;
$totalAdmins = 0;
$totalEvents = 0;

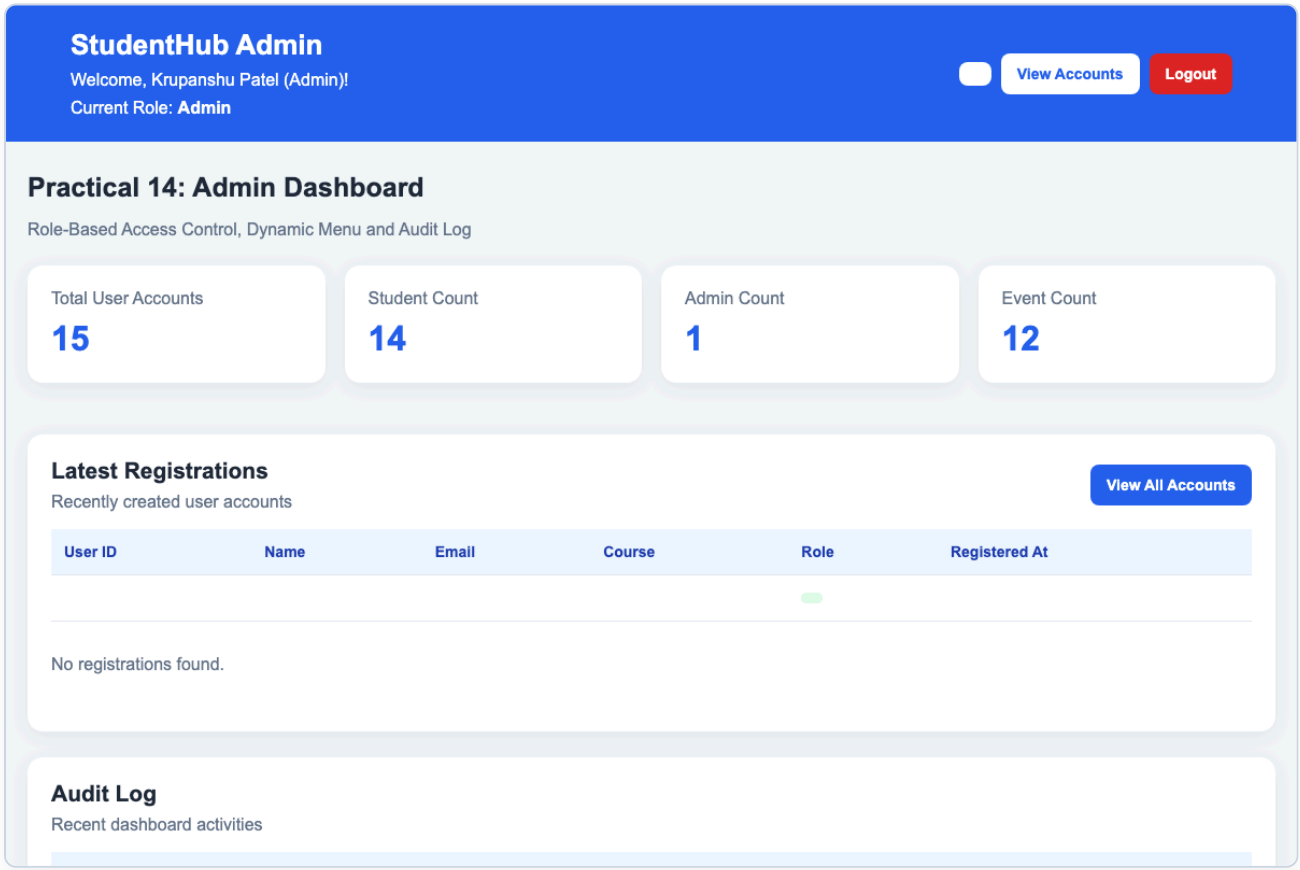
// Query all users
$sql = "
SELECT
    u.user_id,
    u.full_name,
    u.email,
    u.mobile,
    u.course,
    u.year,
    u.gender,
```

```
    u.created_at,
    COALESCE(ur.role, 'student') AS role
FROM users u
LEFT JOIN user_roles ur ON ur.user_id = u.user_id
ORDER BY u.user_id DESC
";

$result = $conn->query($sql);
if ($result) {
    while ($row = $result->fetch_assoc()) {
        $students[] = $row;
    }
} else {
    error_log($conn->error);
    $error = "Unable to load account records.";
}

$totalAccounts = count($students);
foreach ($students as $student) {
    if ($student["role"] === "admin") {
        $totalAdmins++;
    } else {
        $totalStudents++;
    }
}

// Total events...
```



Admin Dashboard with Metrics & Audit Logs Output

Conclusion:

Created an administrative management console featuring dynamic SQL aggregation metrics, audit log traceability for admin actions, and interactive search modals for viewing user profiles.

Practical 15: Final Integration, Testing, Documentation, GitHub Submission, and Local Deployment

Problem Definition:	Integrate all modules into a single deployable StudentHub portal. Verify navigation, responsiveness, authentication, database operations, security checks, AJAX functionality, and documentation. Deploy locally using XAMPP and submit GitHub repository.
CO Mapping:	CO2, CO4, CO5
Key Skills:	Full-stack integration, End-to-end testing, Database exporting, Deployment documentation

Final GitHub Repository: https://github.com/krupanshupatel2347/StudentHub-Portal

Database Schema Dump (studenthub_db.sql - Excerpt):

```
-- ========================================================================
-- Database SQL Dump: studenthub_db
-- Subject: Web Development Frameworks (ITUE203) – Practical 15
-- Student Name: Krupanshu Patel
-- Roll No: 25DCS088
-- University: CHARUSAT / Devang Patel Institute of Advance Technology and Research (DEPSTAR)
-- ========================================================================

CREATE DATABASE IF NOT EXISTS `studenthub_db` CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE `studenthub_db`;

-- Drop existing tables if re-importing
DROP TABLE IF EXISTS `audit_logs`;
DROP TABLE IF EXISTS `remember_tokens`;
DROP TABLE IF EXISTS `user_roles`;
DROP TABLE IF EXISTS `events`;
DROP TABLE IF EXISTS `students`;
DROP TABLE IF EXISTS `users`;

-- 1. Table structure for table `users`
CREATE TABLE `users` (
  `user_id` INT AUTO_INCREMENT PRIMARY KEY,
  `full_name` VARCHAR(100) NOT NULL,
  `email` VARCHAR(100) NOT NULL UNIQUE,
  `mobile` VARCHAR(10) NOT NULL,
  `password` VARCHAR(255) NOT NULL,
  `course` VARCHAR(50) NOT NULL,
  `year` TINYINT NOT NULL,
  `gender` VARCHAR(10) NOT NULL,
  `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 2. Table structure for table `user_roles`
CREATE TABLE `user_roles` (
  `id` INT AUTO_INCREMENT PRIMARY KEY,
  `user_id` INT NOT NULL,
  `role` ENUM('student', 'admin') NOT NULL DE...
```

Integration & Deployment Documentation (README.md - Excerpt):

```
# StudentHub Portal

**Subject:** Web Development Frameworks (ITUE203)
**Semester:** 3rd Semester, A.Y. 2026-27 (ODD)
**Institution:** Devang Patel Institute of Advance Technology and Research (DEPSTAR)
**University:** Charotar University of Science and Technology (CHARUSAT)

**Author / Student Information:**
- **Name:** Krupanshu Patel
- **Roll Number:** 25DCS088
- **GitHub Username:** krupanshupatel2347ok-cmd
- **GitHub Repository:** [https://github.com/krupanshupatel2347/StudentHub-Portal](https://github.
com/krupanshupatel2347/StudentHub-Portal)

---

## 📌 Project Overview
**StudentHub** is an all-in-one full-stack student academic portal developed across the semester f
or **ITUE203 (Web Development Frameworks)**. It encompasses all 15 laboratory practicals into a pr
oduction-grade, coherent web application featuring modern semantic HTML5, responsive CSS Grid and
Flexbox layouts, interactive client-side JavaScript, server-side PHP processing, MySQL database pe
rsistence, session security with Role-Based Access Control (RBAC), and RESTful AJAX/Fetch CRUD API
s.

---

## 🚀 Practical Index & Syllabus Coverage

| Practical | Title | Core Technology / Concepts |
| :---: | :--- | :--- |
| **Prac 1** | Project Initiation, Requirement Analysis, Sitemap, Wireframe & GitHub Setup | Requi
rements, Wireframes, Git/GitHub |
| **Prac 2** | Semantic HTML5 Pages with Ac...
```

Conclusion & Submission:

Successfully integrated all 15 practicals into a unified StudentHub portal. Verified complete navigation flow, responsive UI on all screen sizes, secure session RBAC authentication, database persistence, and AJAX endpoints. The repository has been deployed and committed to GitHub under student **Krupanshu Patel (25DCS088)**.